

## **Discussion**

In this experiment, interrupt handling was implemented using the PIC16F628A microcontroller. The main program continuously toggled the LEDs connected to RA0 and RA1, while the push button connected to RB0/INT was used to trigger an external interrupt.

When the switch was pressed, the microcontroller temporarily stopped the main loop and executed the Interrupt Service Routine. During this interrupt, the LEDs connected to RB1 and RB2 were toggled with a delay. After completing the interrupt task, the program returned to the main loop and continued normal operation.

The PORT pins were configured using TRISA and TRISB, where RA0 and RA1 were set as outputs, RB0 was set as the interrupt input, and RB1 and RB2 were set as outputs. The comparator module was disabled using  $CMCON = 0x07$  to ensure the pins worked properly as digital I/O pins.

This experiment showed how interrupts allow a microcontroller to respond quickly to an external event without continuously checking the switch in the main program. It also helped to understand the importance of interrupt enable bits, interrupt flags, and clearing INTCON.INTF after the interrupt is handled. Overall, the circuit worked as expected and demonstrated the basic use of external interrupts in embedded systems.